Parameter-Efficient Fine-Tuning on IMDB: From Hand-Engineered Features to LoRA

NLP CS60075, Assignment II, Project 9

Group 27
22CS30019 Darapu Adhithya Shiva Kumar Reddy
23EC10099 Avik Ghosh
26CD91R05 Ashutosh Kumar Barnwal

Abstract

We compare three approaches to IMDB movie-review sentiment classification. Task 1 is a classical baseline: TF-IDF over word 1–2 grams with a Random Forest, which reaches 0.8477 test accuracy. Task 2 reproduces LoRA [1] on `bert-base-uncased`. Accuracy rises steadily with rank, from 0.9292 at r=4 to 0.9369 at r=32, against 0.9393 for full fine-tuning, while LoRA trains only 0.14% to 1.07% of the weights. Task 3 stacks the two models with a logistic-regression meta-model fitted on held-out predictions. It reaches 0.9394, a gain of 0.26 points over the best LoRA model (95% CI [+0.13, +0.40], McNemar $p = 0.00018$), which matches full fine-tuning. A joint network that feeds TF-IDF features into the LoRA model end to end does worse (0.9349), because its large TF-IDF branch overfits.

1 Problem and setup

Before pretrained transformers, adapting to a new task meant engineering features by hand. Full fine-tuning replaced that with one general model, but it updates and stores all of BERT's ∼110M weights for every task. LoRA freezes the pretrained model and learns a low-rank update $\Delta W = BA$, with $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times d}$ for a small rank r. Each adapted $d \times d$ matrix then costs only $2dr$ new parameters. Our question is whether hand-engineered features still add anything once a pretrained model is adapted this cheaply. IMDB is a good testbed because its reviews are long, and long documents are where TF-IDF baselines tend to be strongest.

Data. We use `stanfordnlp/imdb` [2], with 25,000 training and 25,000 test reviews. Both splits are balanced at 12,500 positive and 12,500 negative reviews. Cleaning replaces leftover `<br />` tags with spaces and collapses whitespace. Reviews are long: the median is 222 subwords in the training set and 219 in the test set (BERT WordPiece). 13.3% of training reviews and 12.3% of test reviews exceed BERT's 512-subword limit and get truncated.

Train, holdout and test splits. A stacked meta-model must learn from predictions its base models made on reviews they never trained on. Otherwise it sees overconfident in-sample probabilities and learns the wrong trust weights. We therefore split the official training set once, stratified, with seed 42:

- **Core** (22,500 reviews): trains both the Task 1 and Task 2 base models.
- **Holdout** (2,500 reviews): fits the Task 3 meta-model, selects the best LoRA rank and tracks training curves.
- **Test** (the official 25,000 reviews): used only for reported numbers.

Task 2 trains for a fixed 3 epochs and reports the final epoch, so the holdout never selects a checkpoint. For Task 1 we also fit the specified model on all 25,000 training reviews, so a spec-exact baseline number is available.

Metric and reproducibility. The metric is accuracy on the full test set. With 25,000 test reviews, the standard error of an accuracy near 0.93 is about 0.16 points. On the 2,500-review holdout it's about 0.51 points, so small holdout differences are noise. One seed (42) fixes the data split, the Random Forest, model initialisation, data order and the bootstrap.

2 Task 1: TF-IDF and Random Forest

Method. `TfidfVectorizer` uses word 1–2 grams, lowercasing and a 20,000-feature cap, and is fit on training reviews only. The Random Forest has 300 trees, a maximum depth of 30 and at least 5 samples per leaf. There's no pretrained component anywhere.

Model	Train reviews	Holdout accuracy	Test accuracy
TF-IDF + RF (spec-exact)	25,000	n/a	0.8477
TF-IDF + RF (core split, feeds Task 3)	22,500	0.8528	0.8439

Table 1: Task 1 results.

Observations. Word and bigram counts alone classify about 84% to 85% of reviews correctly. That's high because IMDB reviews are long and keep repeating strong cue words such as *worst*, *waste*, *excellent* and ratings like "7/10". The extra 2,500 training reviews add 0.38 points (0.8439 to 0.8477), so the baseline is still slightly data-limited. The Random Forest is also uncertain: its probabilities tend to stay close to 0.5 (see the examples in Section 5), which matters for how Task 3 weights it (Section 4). Its typical failures are the usual ones for a bag of n-grams: negation with long scope, reviews that praise the cast but pan the plot, and sarcasm.

3 Task 2: LoRA reproduction

Method. Following Hu et al. [1], we start from `bert-base-uncased` [3] with a sequence-classification head and freeze all base weights. LoRA matrices go into the query and value projections of all 12 layers, with $r \in \{4, 8, 16, 32\}$, $\alpha = 2r$ and LoRA dropout 0.1. The classification head stays trainable. Training uses AdamW at a learning rate of 3×10^{-4}, batch size 16, weight decay 0.01 and 3 epochs. The learning rate decays linearly with no warmup, gradients are clipped at 1.0 and training runs in fp16 mixed precision on a T4 GPU. The full fine-tuning reference uses the same checkpoint with a learning rate of 2×10^{-5} and an effective batch of 16 (8×2 gradient accumulation). Inputs are truncated to 512 subwords.

Model	Trainable params	% of total	Holdout acc	Test acc	Train time (min)
LoRA r=4	148,994	0.136	0.9212	0.9292	26.2
LoRA r=8	296,450	0.270	0.9196	0.9327	26.3
LoRA r=16	591,362	0.537	0.9272	0.9351	26.3
LoRA r=32	1,181,186	1.067	0.9288	**0.9369**	26.3
Full fine-tuning	109,483,778	100	0.9324	**0.9393**	31.5

Table 2: Task 2 results. Parameter counts include the 1,538-parameter classification head. The best rank by holdout accuracy is r=32.

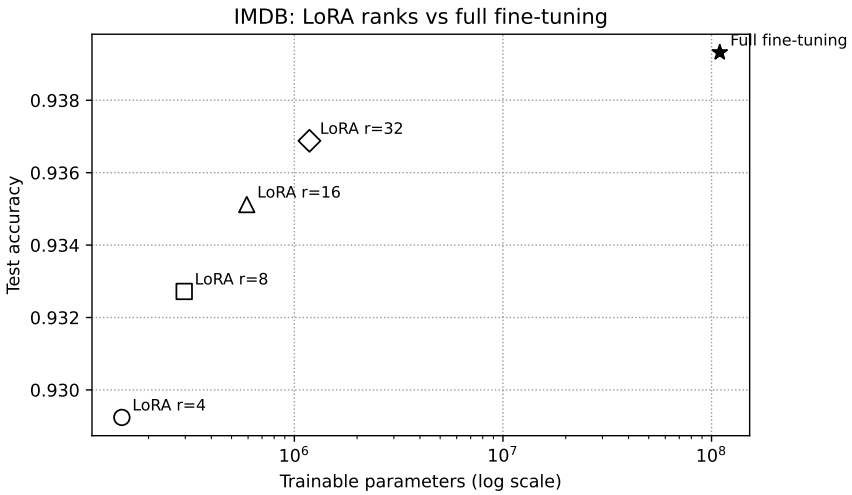


Figure 1: Trainable parameters (log scale) against test accuracy.

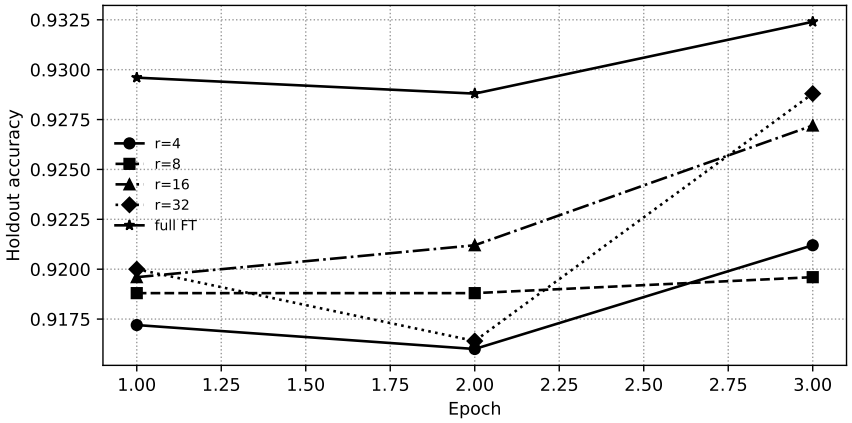


Figure 2: Holdout accuracy after each epoch.

Observations.

- **LoRA recovers almost all of full fine-tuning.** Measured against the Task 1 baseline trained on the same 22,500 reviews, r=32 closes 97.5% of the gap to full fine-tuning: $(0.9369 - 0.8439)/(0.9393 - 0.8439)$. It trains 93 times fewer parameters. Even r=4 closes 89% of the gap with 735 times fewer parameters.
- **Accuracy keeps rising with rank, but each doubling buys less.** The gains are +0.35, +0.24 and +0.18 points for $r = 4 \to 8$, $8 \to 16$ and $16 \to 32$. That supports the paper's claim that the useful update has low intrinsic rank: a 1% parameter budget already lands within 0.24 points of full fine-tuning.
- **Holdout and test disagree slightly on ranks 4 and 8**. Rank 8 is lower on the holdout but higher on test. The difference (0.16 points) is far below the holdout's 0.51-point standard error, so the test ranking is the one to trust.
- **All models were still improving at epoch 3** (Figure 2). Ranks 4 and 32 dip at epoch 2 and recover in epoch 3, as the linearly decaying learning rate approaches zero. More epochs or a tuned schedule might add a little accuracy.
- **LoRA is cheaper per step as well as per checkpoint.** Each LoRA run took about 26 minutes against 31.5 minutes for full fine-tuning, and a LoRA adapter is a few megabytes against about 440 MB for a fully fine-tuned model.

Comparison with the paper. Hu et al. report GLUE results for RoBERTa, DeBERTa and GPT models rather than BERT on IMDB, so there's no number to match exactly. Their central claim is that a low-rank update matches or nearly matches full fine-tuning at a tiny fraction of the trainable parameters. Our results reproduce that shape. The remaining 0.24-point gap is plausibly explained by three choices: we adapt only the query and value matrices, we train for a fixed 3 epochs, and we didn't tune the LoRA learning rate.

4 Task 3: Classical-feature-augmented LoRA

Method. Following stacked generalisation [4], the meta-model is a logistic regression (C=1) with two inputs: the log-odds of the Random Forest's positive-class probability and the log-odds of the best LoRA model's (r=32) probability. Log-odds is the natural input scale for logistic regression. We also report a version on raw probabilities and a plain probability average. The meta-model is fit on the 2,500 holdout reviews and evaluated once on the test set. We test the difference from LoRA alone with a paired bootstrap (2,000 resamples) and an exact McNemar test.

Model	Test accuracy
Task 1: TF-IDF + RF (core split)	0.8439
Task 2: LoRA r=32	0.9369
Probability average of Task 1 and Task 2	0.9372
Meta-model on raw probabilities	0.9374
Joint TF-IDF + LoRA network (Section 4.1)	0.9349
Task 3: meta-model on log-odds	**0.9394**
Reference: full fine-tuning (Task 2)	0.9393

Table 3: Task 3 against the base models. The meta-model's 5-fold cross-validated accuracy on the holdout is 0.9292. Task 3 minus Task 2: +0.26 points, 95% bootstrap CI [+0.13, +0.40], McNemar $p = 0.000184$. Learned coefficients: 1.796 on the RF log-odds and 0.499 on the LoRA log-odds.

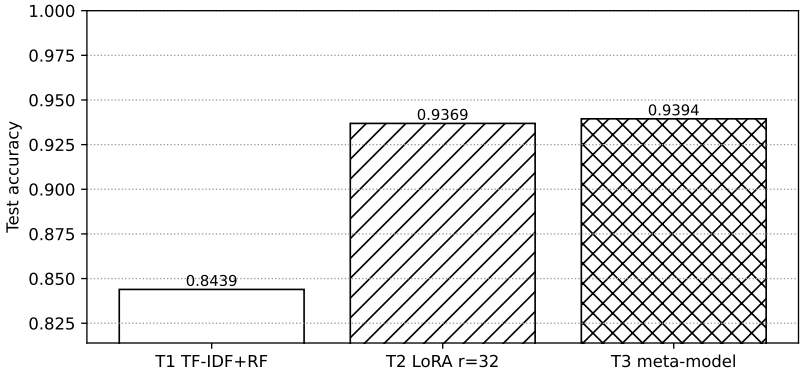


Figure 3: Test accuracy of the best model from each task.

Test subset	n	RF	LoRA	Meta
All test reviews	25,000	0.8439	0.9369	0.9394
Longer than 512 subwords (BERT truncated)	3,086	0.8377	0.9109	0.9145
RF and LoRA disagree	3,678	0.1841	0.8159	0.8336

Table 4: Accuracy on the subsets where the two base models differ most.

Observations.

- **Word counts do add information BERT misses.** The gain is small but statistically clear: +0.26 points, or about 65 reviews net, with a confidence interval that excludes zero. The stacked model, whose neural part trains about 1.2M parameters, matches full fine-tuning (0.9394 vs. 0.9393), which trains 109M.
- **How the stack combines the models matters.** A plain average (0.9372) and the raw-probability meta-model (0.9374) barely beat LoRA alone. The log-odds meta-model gains about five times as much as the raw-probability version. LoRA's probabilities sit near 0 or 1, so averaging them with the Random Forest's near-0.5 outputs changes almost nothing. The log-odds scale lets the meta-model rescale each model separately.
- **The coefficients don't mean RF matters more.** The RF coefficient (1.796) is larger than LoRA's (0.499) because RF log-odds are small: its probabilities stay close to 0.5. When LoRA is confident, at $p = 0.999$, its log-odds term is $0.499 \times 6.9 \approx 3.4$. That overrides anything the Random Forest can contribute. In effect the Random Forest acts as a tie-breaker when LoRA is unsure.
- **All of the gain comes from disagreements.** The two models disagree on 3,678 test reviews (14.7%). LoRA is right on 81.6% of these and the Random Forest on 18.4%. The meta-model is right on 83.4%, so it learns to side with the Random Forest on a useful subset of them.
- **Truncation hurts BERT, and the stack helps a little there.** On the 3,086 truncated reviews, LoRA drops to 0.9109 from 0.9369 overall, while the Random Forest drops much less (0.8377 vs. 0.8439). The meta-model's gain on truncated reviews (+0.36 points) is somewhat larger than overall (+0.26). Seeing the whole review is part of what TF-IDF contributes, but most of the gain still comes from reviews BERT reads in full.

4.1 Alternative: joint TF-IDF + LoRA model

Method. The spec also allows feeding the classical features into the model directly. We build a joint network with two branches. The first is the same LoRA-adapted BERT (r=32, query and value, α=64, dropout 0.1), from which we take the `[CLS]` vector (768 dimensions). The second is the 20,000-dimensional TF-IDF vector from the Task 1 vectorizer (fit on the core split only), projected to 256 dimensions by a linear layer with ReLU and dropout 0.3. The two vectors are concatenated, passed through LayerNorm and dropout 0.1, and classified by a linear layer. The whole model trains end to end with cross-entropy on the 22,500 core reviews, with the same optimiser, learning rate, batch size and 3 epochs as Task 2.

Model	Trainable params	Test accuracy
LoRA r=32 alone (Task 2)	1,181,186	0.9369
Joint TF-IDF + LoRA r=32	6,304,002	0.9349
Stacked meta-model (Task 3, main)	1,181,186 + RF + 3	**0.9394**

Table 5: Joint model against LoRA alone and the stack. The joint model's parameters break down as 1,179,648 LoRA weights, 5,120,256 in the TF-IDF projection, and 4,098 in the fusion head. Training took 27.9 minutes.

Observations.

- **Fusing the features inside the network made it worse.** The joint model scored 0.9349, which is 0.20 points below LoRA alone and 0.45 points below the stack. We didn't run a paired significance test for this variant, so we read the gap to LoRA as "no improvement" rather than a confirmed loss.
- **The TF-IDF branch overfits.** The running training loss reached about 0.25 by the end of epoch 1, 0.105 in epoch 2 and 0.043 in epoch 3. The projection layer holds 5.1M of the 6.3M

trainable weights, so 81% of the model's capacity sits on sparse word features. With 22,500 reviews and 20,000 input dimensions, that branch can memorise which words go with which label in the training set. It gives the network a lexical shortcut that doesn't carry over to new reviews.

- **Late fusion beats early fusion here.** The stack keeps the two models separate and learns only three numbers on held-out data, so neither model can overfit to the other. The joint model has to learn how to weight TF-IDF against BERT from the same training reviews it's memorising. A smaller projection, stronger dropout on the TF-IDF branch, or an early stop on the holdout would be the first things to try.

4.2 Alternative: out-of-fold (OOF) stacking (link to another notebook)

Motivation. The main stack in Table 3 fits its meta-model on only 2,500 holdout reviews, and the base models never see those reviews in training. That is clean, but it costs us twice: the meta-model learns from a small sample, and the base models give up 10% of the training data. Out-of-fold stacking is the textbook way around this. Every one of the 25,000 training reviews gets a prediction from a model that never trained on it, so the meta-model can learn from all of them, and the final base models can be trained on the full set. We tried this as a separate experiment, in a second notebook that deliberately differs from the main one (link to another Colab notebook).

Setup. We run a stratified 5-fold split over the 25,000 official training reviews (seed 42). In each fold, a fresh TF-IDF vectorizer and Random Forest are fit on the other four folds (20,000 reviews), and a fresh LoRA r=32 model is trained for 3 epochs on the same 20,000 reviews. Both then predict the 5,000 reviews that were left out. After five folds, every training review has one RF probability pair and one LoRA probability pair, none of them in-sample. A logistic regression with default settings is fit on these four numbers, using all 25,000 labels. For the test set, we retrain the RF and a LoRA r=32 model once on the full 25,000 reviews and feed their test probabilities to that meta-model. The folds use the same hyperparameters as Task 2, with no checkpoint selection, so nothing is tuned on the validation folds.

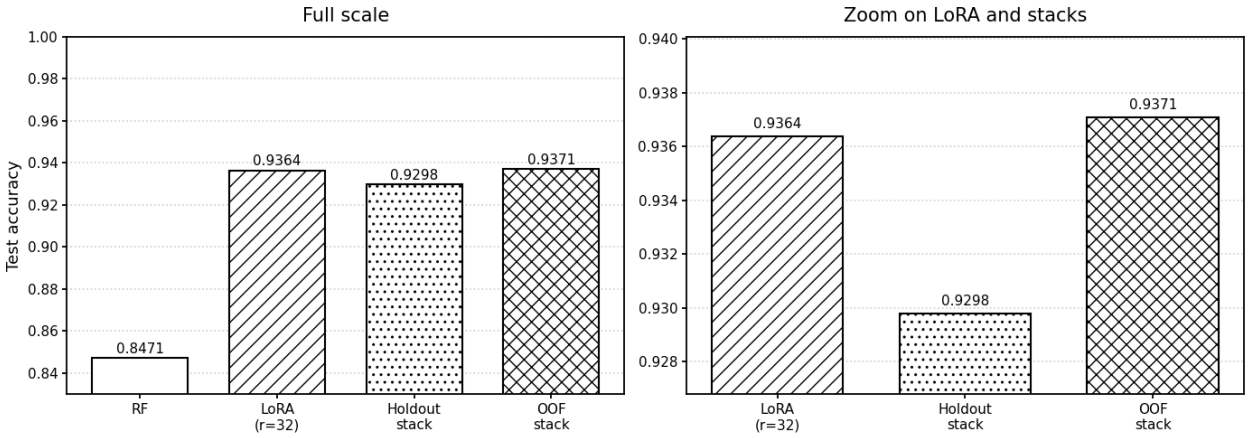


Figure 4: Test accuracy of the Random Forest, LoRA r=32, the holdout stack and the OOF stack in the second notebook. The right panel zooms in on the three close models.

Model (second notebook)	Test accuracy
TF-IDF + RF (trained on 25,000)	0.8471
LoRA r=32 (trained on 25,000)	0.9364
Holdout stack (2,500 reviews, raw probabilities)	0.9298
OOF stack (25,000 reviews, raw probabilities)	**0.9371**

Table 6: OOF stacking against the holdout stack and the base models.

Results. The OOF stack reaches 0.9371. That is 0.73 points above the holdout stack (0.9298) trained in the same notebook, and 0.07 points above LoRA alone (0.9364). The first gap is large, but the comparison is not perfectly fair: the holdout stack is built on the LoRA model trained on 22,500 reviews (0.9330 on test), while the OOF stack uses base models trained on all 25,000. So part of the gap simply reflects more training data going into the base models.

Observations.

- **The gain over LoRA is within noise.** Seven hundredths of a point is about 18 reviews out of 25,000, well under the 0.16-point standard error of a single test accuracy. We did not run a paired test on this variant, so we treat it as "roughly equal to LoRA", not as an improvement.
- **It does not beat the main stack.** The log-odds meta-model in Table 3 reached 0.9394. The OOF stack feeds raw probabilities to the meta-model, and our main experiments already showed that raw probabilities waste most of the Random Forest's signal because LoRA's outputs sit near 0 or 1. Having ten times more meta-training data did not fix that. The input scale mattered more than the amount of data.
- **The cost is high.** OOF stacking needs six LoRA trainings (five folds plus the final model) instead of one, roughly six times the GPU time of the holdout approach, for a result that is no better than the cheaper log-odds stack.
- **Absolute numbers differ slightly from the main notebook.** Here the RF scores 0.8471 against 0.8477, and LoRA 0.9364 against 0.9369. This notebook does not strip `<br />` tags and pads every input to 512 subwords, and fp16 training is not bit-for-bit repeatable. We kept the two notebooks separate on purpose and compare methods only within each one.

Takeaway. OOF stacking is the right idea in principle, and it fixed the holdout stack's weakness in the second notebook. But with LoRA's near-saturated probabilities, an OOF meta-model on raw probabilities ends up level with LoRA alone. The log-odds holdout stack in the main notebook remains our best Task 3 result. A log-odds OOF stack is the obvious next experiment, though we have not run it.

5 Qualitative examples

Texts are truncated to 300 characters. Probabilities are for the positive class.

Task 2: confident LoRA errors.

- *Gold negative,* $p_{pos} = 0.999$. "This movie was pure genius. John Waters is brilliant. It is hilarious and I am not sick of it even after seeing it about 20 times since I bought it a few months ago. The acting is great, although Ricki Lake could have been better. And Johnny Depp is magnificent. He is such a beautiful man and a very"
- *Gold negative,* $p_{pos} = 0.999$. "Lois Weber's film "Hypocrites" was and still kind of is a very bold and daring film. I enjoyed it and was very impressed by the filming and story of it. The priest sees the hypocrisy of the people in his church and tries to show them the "naked" truth. The people are appalled when he reveals the nak"

Both reviews read as clearly positive, yet both carry negative labels. The most confident "errors" look like label noise in the dataset rather than model mistakes. That suggests the accuracy ceiling on IMDB sits somewhat below 100%.

Task 2: correct but unsure.

- *Gold positive,* $p_{pos} = 0.500$. "After seeing the trailer it was an easy decision not to see this film. I mean, I don't care for stupid "stoner comedies." I'm sure it was also an easy choice for a lot of people to get together, smoke a bowl and go check out this flick with the guy from The Simpsons and some guy named "Billy Bob." S"
- *Gold positive,* $p_{pos} = 0.502$. "Watching this hilariously retro but very entertaining career girl tale, I was floored by Joan Crawford's first appearance. All I could think initially was, "My God, it's the same face as Michael Jackson in his notorious booking photo!" About 34 minutes into the movie, Diane Baker and Hope Lange get"

The first review opens with negative framing and turns positive later. The second mixes praise with mockery. These contrastive reviews sit right at the decision boundary.

Task 3: the meta-model fixed a LoRA error.

- *Gold negative;* $p_{rf} = 0.392$, $p_{lora} = 0.757$, $p_{meta} = 0.438$. "First of all, I wasn't sure who this film was aimed at - it seemed like a story for kids but had stuff in it kids wouldn't understand and find boring. There wasn't really much to it, Bruce Willis wasn't stretched as an actor at all. He did a lot of glancing to the side with that half smile of his -"
- *Gold negative;* $p_{rf} = 0.420$, $p_{lora} = 0.593$, $p_{meta} = 0.396$. "The people who bash this movie were looking for it to be as cool and slick as the first one, which this isn't. This movie was supposed to be the complete opposite as Ocean's 11. This has been said by a lot of the cast members and also the director, Steven Soderbergh. Ocean's 12, while it did lack a"

Task 3: the meta-model broke a correct LoRA call.

- *Gold negative;* $p_{rf} = 0.578$, $p_{lora} = 0.277$, $p_{meta} = 0.514$. "An obvious vanity press for Julie in her first movie with Blake. Let's see. Where do we begin. She is a traitor during a world war; she redeems that by falling in love; her friends (who are presumably patriots because they are German citizens) are expendable and must die; and she winds up as a heroi"
- *Gold negative;* $p_{rf} = 0.566$, $p_{lora} = 0.333$, $p_{meta} = 0.525$. "This movie should have been billed as three movie-summaries linked together to form a full-length feature film (including lots of shots of people slowly walking down dark corridors and streets). BE WARNED! The first hour of this movie is simply a re-hash of the first two Aztec movies as told by the"

The pattern is consistent. The meta-model only overrides LoRA when LoRA is unsure (probabilities between about 0.25 and 0.75) and the Random Forest leans the other way. The broken cases are sarcastic plot summaries. Their words sound neutral or positive ("falls in love", "heroine", "full-length feature") while the tone is negative, which fools a bag of words.

6 Conclusions

- **LoRA is the practical winner.** At r=32 it lands within 0.24 points of full fine-tuning while training 1.07% of the weights, and it trains faster.
- **Hand-engineered features still help, a little.** A log-odds stack of TF-IDF + RF and LoRA adds a significant 0.26 points and matches full fine-tuning. The Random Forest contributes mainly as a tie-breaker on reviews LoRA is unsure about, partly because it sees text past BERT's 512-subword cut-off.

- **How you combine them matters more than whether you combine them.** Late fusion (the stack) helps, while early fusion (the joint network) scored below LoRA alone, because its 5.1M-parameter TF-IDF branch memorised the training set.
- **Limitations.** We ran one seed per configuration, didn't tune the LoRA learning rate, adapted only the query and value matrices, and didn't regularise or early-stop the joint model. Inputs were truncated at 512 subwords, and the base models trained on 90% of the training data to leave a clean holdout for stacking.

7 Contributions

- **Avik Ghosh (23EC10099).** Task 1, the project documentation, and the presentation slides.
- **Darapu Adhithya Shiva Kumar Reddy (22CS30019).** Task 3, and the notebook formatting, including the Markdown cells.
- **Ashutosh Kumar Barnwal (26CD91R05).** Task 2, and the alternative methods explored in Task 3, such as out-of-fold (OOF) stacking, which did not improve the score.

8 Reproducibility

The notebook `IMDB_Assignment.ipynb` runs end to end on Google Colab with a GPU runtime. Run the setup and data cells first in every session. Each task then runs on its own, in any session, because every task saves its outputs to Drive and Task 3 only loads them. Task 2 skips ranks that are already trained, and `QUICK_MODE` runs the whole pipeline on a small subsample in a few minutes. The dataset is `stanfordnlp/imdb` from the Hugging Face Hub, with an automatic fallback to the original Stanford tarball. The pretrained checkpoint is `bert-base-uncased`. We have also provided the requirements.txt inside code directory.

References

[1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen. LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022, arXiv:2106.09685.

[2] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, C. Potts. Learning Word Vectors for Sentiment Analysis. ACL 2011.

[3] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL 2019.

[4] D. H. Wolpert. Stacked Generalization. Neural Networks 5(2), 1992.